Correlation and Calibration - Fertilizer Recommendations

A Bayesian Approach

Gustavo A. Roa

2026-03-27

Table of contents

1 Overview **2**

2 Setup **2**
- 2.1 Prerequisites . . . 2
- 2.2 Check the Installation . . . 2
- 2.3 Required Packages . . . 3
- 2.4 Import Data . . . 3

3 Bayesian Estimation of Critical Soil Test Values **4**
- 3.1 Bayesian Quadratic-Plateau Model . . . 4
- 3.2 Bayesian Linear-Plateau Model . . . 10
- 3.3 Bayesian Exponential (Mitscherlich) Model . . . 16

4 Bayesian Model Averaging **22**
- 4.1 Estimate Marginal Likelihoods and Posterior Model Probabilities . . . 22
- 4.2 Construct the Model-Averaged CSTV Posterior . . . 23
- 4.3 Results Table for CSTV . . . 25

5 Bayesian Calibration of Fertilizer Recommendations **26**
- 5.1 Summarize the CSTV Posterior . . . 26
- 5.2 Calculate the Deficiency Index . . . 27
- 5.3 Estimate the Maximum Fertilizer Rate . . . 27
- 5.4 Summarize the Maximum Fertilizer Rate . . . 32
- 5.5 Calculate Fertilizer Recommendations . . . 32

6 Summary of Results **34**
6.1 Critical Soil Test Value 34
6.2 Fertilizer Recommendation 35

1 Overview

Soil-test correlation and calibration are essential for developing fertilizer recommendations.This tutorial demonstrates how to estimate CSTVs using Bayesian nonlinear regression and Bayesian model averaging (BMA). It also illustrates how the resulting CSTV posterior distribution can be used to calibrate phosphorus fertilizer recommendations and account for uncertainty.

2 Setup

2.1 Prerequisites

Before starting, make sure that R and RStudio are installed. This tutorial uses `brms` to fit Bayesian nonlinear regression models through Stan, which requires a working C++ toolchain to compile the statistical models.

- **R:** The statistical programming environment.
- **RStudio (recommended):** An integrated development environment for working with R and Quarto.
- **C++ toolchain:** Required by Stan to compile the models used by `brms`. Windows users can install Rtools; macOS users need the Xcode Command Line Tools; Linux users need the appropriate compiler and development tools.

For installation instructions, consult the official documentation:

- R
- RStudio
- Rtools for Windows

2.2 Check the Installation

Check the R version and verify that a C++ compiler is available.

```
# Check the installed R version
R.version.string
```

```
[1] "R version 4.6.1 (2026-06-24 ucrt)"
```

```
# Check whether R can find a C++ compiler
Sys.which(c("make", "g++", "clang++"))
```

```
                                     make
          "C:\\rtools45\\usr\\bin\\make.exe"
                                      g++
"C:\\rtools45\\X86_64~1.POS\\bin\\G__~1.EXE"
                                  clang++
                                       ""
```

2.3 Required Packages

The following packages are used for data manipulation, Bayesian modeling, and Bayesian model averaging.

```
# Data manipulation
library(tidyverse)

# Data import
library(readxl)

# Bayesian modeling
library(Rcpp)
library(brms)

# Marginal likelihood estimation and Bayesian model averaging
library(bridgesampling)
```

2.4 Import Data

The example dataset contains the soil-test phosphorus (P) measurements, relative grain yield, and optimum fertilizer rates used in this tutorial. These data are used to estimate the Critical Soil Test Value (CSTV) and calculate phosphorus fertilizer recommendations.

The dataset is provided as an Excel file named dataset_example.xlsx, with the data stored in a worksheet named data. Place the file in the same folder as this tutorial before running the code.

```r
# Import the experimental dataset
data_example <- read_excel(
  "./dataset_example.xlsx",
  sheet = "data"
)

# Inspect the dataset structure
str(data_example)
```

```
tibble [34 x 4] (S3: tbl_df/tbl/data.frame)
 $ loc_num                : num [1:34] 1 2 3 4 5 6 7 8 9 10 ...
 $ relative_yield         : num [1:34] 91 97 85 100 97 100 95 91 95 96 ...
 $ soil_test              : num [1:34] 23 46 21 40 26 47 22 11 20 43 ...
 $ optimun_fertilizer_rate: num [1:34] 0 0 12 0 0 0 0 0 0 0 ...
```

The dataset must include the following variables:

soil_test: Soil-test phosphorus concentration.

relative_yield: Relative grain yield used to fit the soil-test response models.

optimun_fertilizer_rate: Optimum fertilizer rate used to estimate the maximum fertilizer rate.

Use the same measurement units as those in the provided dataset throughout the analysis. The soil-test P values and fertilizer rates should not be converted unless the corresponding model equations and results are adjusted accordingly.

The following sections use these variables to fit the Bayesian response models, estimate the CSTV, and calculate phosphorus fertilizer recommendations.

3 Bayesian Estimation of Critical Soil Test Values

3.1 Bayesian Quadratic-Plateau Model

The quadratic-plateau (QP) model describes a quadratic increase in relative yield followed by a plateau. The parameter x_s represents the soil-test phosphorus concentration at which the response reaches the plateau and is used as the CSTV estimate.

```r
# Define the Bayesian quadratic-plateau model
bf_QP <- brms::bf(
  relative_yield ~ a +
    b * fmin(soil_test, xs) -
    (b / (2 * xs)) * fmin(soil_test, xs)^2,
  a ~ 1,
  b ~ 1,
  xs ~ 1,
  nl = TRUE
)

# Specify priors for the model parameters
priors_QP <- c(
  brms::set_prior(
    "normal(58.3, 0.5*58.3)",
    nlpar = "a",
    lb = 0
  ),
  brms::set_prior(
    "normal(3.8, 0.5*3.8)",
    nlpar = "b",
    lb = 0
  ),
  brms::set_prior(
    "normal(21.4, 0.5*21.4)",
    nlpar = "xs",
    lb = 0
  ),
  brms::set_prior(
    "student_t(3, 0, 10)",
    class = "sigma"
  )
)

# Fit the Bayesian quadratic-plateau model
fit_QP_bayes <- brms::brm(
  formula = bf_QP,
  data = data_example,
  prior = priors_QP,
  chains = 5,
  iter = 3000,
  warmup = 1000,
```

```
  cores = 1,
  seed = 555,
  save_pars = brms::save_pars(all = TRUE),
  control = list(
    adapt_delta = 0.99,
    max_treedepth = 15
  )
)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 7.8e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.78 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.733 seconds (Warm-up)
Chain 1:                1.788 seconds (Sampling)
Chain 1:                2.521 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.5e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.15 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
```

```
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.739 seconds (Warm-up)
Chain 2:                2.173 seconds (Sampling)
Chain 2:                2.912 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 1.6e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.16 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 3: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 3: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 3: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 3: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 3: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 3: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 3: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 3: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 3: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 3: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 3: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.792 seconds (Warm-up)
Chain 3:                3.099 seconds (Sampling)
Chain 3:                3.891 seconds (Total)
Chain 3:
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 2.4e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.24 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 4: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 4: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 4: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 4: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 4: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 4: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 4: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 4: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 4: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 4: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 4: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 1.075 seconds (Warm-up)
Chain 4:                2.228 seconds (Sampling)
Chain 4:                3.303 seconds (Total)
Chain 4:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 5).
Chain 5:
Chain 5: Gradient evaluation took 2e-05 seconds
Chain 5: 1000 transitions using 10 leapfrog steps per transition would take 0.2 seconds.
Chain 5: Adjust your expectations accordingly!
Chain 5:
Chain 5:
Chain 5: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 5: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 5: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 5: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 5: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 5: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 5: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 5: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 5: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 5: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 5: Iteration: 2800 / 3000 [ 93%]  (Sampling)
```

```
Chain 5: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 5:
Chain 5:  Elapsed Time: 1.079 seconds (Warm-up)
Chain 5:                2.115 seconds (Sampling)
Chain 5:                3.194 seconds (Total)
Chain 5:
```

```r
# Summary model results
summary(fit_QP_bayes)
```

```
 Family: gaussian
  Links: mu = identity
Formula: relative_yield ~ a + b * fmin(soil_test, xs) - (b/(2 * xs)) * fmin(soil_test, xs)^2
         a ~ 1
         b ~ 1
         xs ~ 1
   Data: data_example (Number of observations: 34)
  Draws: 5 chains, each with iter = 3000; warmup = 1000; thin = 1;
         total post-warmup draws = 10000

Regression Coefficients:
             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
a_Intercept     72.19      4.87    61.13    79.90 1.00     2261     2583
b_Intercept      2.12      1.20     0.76     5.32 1.00     1958     2638
xs_Intercept    23.33      8.84     9.95    42.65 1.00     2283     3333

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     7.37      0.97     5.77     9.55 1.00     3548     3928

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
# Extract posterior draws of the CSTV parameter
xs_QP <- brms::as_draws_df(
  fit_QP_bayes
)$b_xs_Intercept

# Calculate the posterior median and 95% credible interval
xs_QP_hat <- median(xs_QP)
```

```r
xs_QP_ci <- quantile(
  xs_QP,
  probs = c(0.025, 0.975)
)

# Display the CSTV estimate and uncertainty
cat("\nBayesian Quadratic-Plateau Model\n")
```

```
Bayesian Quadratic-Plateau Model
```

```r
cat(
  "CSTV estimate:",
  round(xs_QP_hat, 2),
  "\n"
)
```

```
CSTV estimate: 21.9
```

```r
cat(
  "95% Credible Interval:",
  round(xs_QP_ci[1], 2),
  "-",
  round(xs_QP_ci[2], 2),
  "\n"
)
```

```
95% Credible Interval: 9.95 - 42.65
```

3.2 Bayesian Linear-Plateau Model

The linear-plateau (LP) model describes a linear increase in relative yield followed by a plateau. As in the QP model, x_s represents the soil-test phosphorus concentration at which the response reaches the plateau.

```r
# Define the Bayesian linear-plateau model
bf_LP <- brms::bf(
  relative_yield ~ a + b * fmin(soil_test, xs),
  a ~ 1,
```

```
  b ~ 1,
  xs ~ 1,
  nl = TRUE
)

# Specify priors for the model parameters
priors_LP <- c(
  brms::set_prior(
    "normal(64.5, 0.5*64.5)",
    nlpar = "a",
    lb = 0
  ),
  brms::set_prior(
    "normal(2.25, 0.5*2.25)",
    nlpar = "b",
    lb = 0
  ),
  brms::set_prior(
    "normal(15, 0.5*15)",
    nlpar = "xs",
    lb = 0
  ),
  brms::set_prior(
    "student_t(3, 0, 10)",
    class = "sigma"
  )
)

# Fit the Bayesian linear-plateau model
fit_LP_bayes <- brms::brm(
  formula = bf_LP,
  data = data_example,
  prior = priors_LP,
  chains = 5,
  iter = 3000,
  warmup = 1000,
  cores = 1,
  seed = 555,
  save_pars = brms::save_pars(all = TRUE),
  control = list(
    adapt_delta = 0.99,
    max_treedepth = 15
```

```
  )
)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.000127 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.27 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 1.063 seconds (Warm-up)
Chain 1:                1.846 seconds (Sampling)
Chain 1:                2.909 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.5e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.15 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
```

```
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.708 seconds (Warm-up)
Chain 2:                1.026 seconds (Sampling)
Chain 2:                1.734 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 2.7e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.27 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 3: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 3: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 3: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 3: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 3: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 3: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 3: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 3: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 3: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 3: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 3: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.613 seconds (Warm-up)
Chain 3:                1.29 seconds (Sampling)
Chain 3:                1.903 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 1.6e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.16 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
```

```
Chain 4:
Chain 4: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 4: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 4: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 4: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 4: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 4: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 4: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 4: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 4: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 4: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 4: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 4: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.7 seconds (Warm-up)
Chain 4:                1.382 seconds (Sampling)
Chain 4:                2.082 seconds (Total)
Chain 4:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 5).
Chain 5:
Chain 5: Gradient evaluation took 2.4e-05 seconds
Chain 5: 1000 transitions using 10 leapfrog steps per transition would take 0.24 seconds.
Chain 5: Adjust your expectations accordingly!
Chain 5:
Chain 5:
Chain 5: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 5: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 5: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 5: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 5: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 5: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 5: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 5: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 5: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 5: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 5: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 5: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 5:
Chain 5:  Elapsed Time: 0.693 seconds (Warm-up)
Chain 5:                1.566 seconds (Sampling)
Chain 5:                2.259 seconds (Total)
Chain 5:
```

```r
# Summary(model results
summary(fit_LP_bayes)
```

```
 Family: gaussian
  Links: mu = identity
Formula: relative_yield ~ a + b * fmin(soil_test, xs)
         a ~ 1
         b ~ 1
         xs ~ 1
   Data: data_example (Number of observations: 34)
  Draws: 5 chains, each with iter = 3000; warmup = 1000; thin = 1;
         total post-warmup draws = 10000

Regression Coefficients:
             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
a_Intercept     74.88      3.88    66.26    81.56 1.00     2321     2665
b_Intercept      1.25      0.65     0.48     3.05 1.00     1908     2657
xs_Intercept    16.52      6.11     7.22    31.10 1.00     2117     3565

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     7.40      0.97     5.77     9.58 1.00     3522     4289

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
# Extract posterior draws of the CSTV parameter
xs_LP <- brms::as_draws_df(
  fit_LP_bayes
)$b_xs_Intercept

# Calculate the posterior median and 95% credible interval
xs_LP_hat <- median(xs_LP)

xs_LP_ci <- quantile(
  xs_LP,
  probs = c(0.025, 0.975)
)

# Display the CSTV estimate and uncertainty
cat("\nBayesian Linear-Plateau Model\n")
```

```
Bayesian Linear-Plateau Model
```

```
cat(
  "CSTV estimate:",
  round(xs_LP_hat, 2),
  "\n"
)
```

```
CSTV estimate: 15.46
```

```
cat(
  "95% Credible Interval:",
  round(xs_LP_ci[1], 2),
  "-",
  round(xs_LP_ci[2], 2),
  "\n"
)
```

```
95% Credible Interval: 7.22 - 31.1
```

3.3 Bayesian Exponential (Mitscherlich) Model

The exponential model describes an asymptotic increase in relative yield as soil-test phosphorus increases. The parameter x_s represents the soil-test phosphorus concentration corresponding to 95% of the asymptotic response.

```
# Define the Bayesian exponential model
bf_EXP <- brms::bf(
  relative_yield ~ a -
    b * exp(
      -(log(b / (0.05 * a)) / xs) * soil_test
    ),
  a ~ 1,
  b ~ 1,
  xs ~ 1,
  nl = TRUE
)

# Specify priors for the model parameters
```

```r
priors_EXP <- c(
  brms::set_prior(
    "normal(100, 0.5*100)",
    nlpar = "a",
    lb = 0
  ),
  brms::set_prior(
    "normal(55, 0.5*55)",
    nlpar = "b",
    lb = 0
  ),
  brms::set_prior(
    "normal(21.4, 0.5*21.4)",
    nlpar = "xs",
    lb = 0
  ),
  brms::set_prior(
    "student_t(3, 0, 10)",
    class = "sigma"
  )
)

# Fit the Bayesian exponential model
fit_EXP_bayes <- brms::brm(
  formula = bf_EXP,
  data = data_example,
  prior = priors_EXP,
  chains = 5,
  iter = 3000,
  warmup = 1000,
  cores = 1,
  seed = 555,
  save_pars = brms::save_pars(all = TRUE),
  control = list(
    adapt_delta = 0.99,
    max_treedepth = 15
  )
)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
```

```
Chain 1: Gradient evaluation took 0.000103 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.03 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.551 seconds (Warm-up)
Chain 1:                0.726 seconds (Sampling)
Chain 1:                1.277 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.7e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.17 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
```

```
Chain 2:  Elapsed Time: 0.567 seconds (Warm-up)
Chain 2:                0.975 seconds (Sampling)
Chain 2:                1.542 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 2.1e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.21 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 3: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 3: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 3: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 3: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 3: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 3: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 3: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 3: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 3: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 3: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 3: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.455 seconds (Warm-up)
Chain 3:                1.269 seconds (Sampling)
Chain 3:                1.724 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 1.9e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.19 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 4: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 4: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 4: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 4: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 4: Iteration: 1300 / 3000 [ 43%]  (Sampling)
```

```
Chain 4: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 4: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 4: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 4: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 4: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 4: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.433 seconds (Warm-up)
Chain 4:                1.136 seconds (Sampling)
Chain 4:                1.569 seconds (Total)
Chain 4:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 5).
Chain 5:
Chain 5: Gradient evaluation took 3.8e-05 seconds
Chain 5: 1000 transitions using 10 leapfrog steps per transition would take 0.38 seconds.
Chain 5: Adjust your expectations accordingly!
Chain 5:
Chain 5:
Chain 5: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 5: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 5: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 5: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 5: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 5: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 5: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 5: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 5: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 5: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 5: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 5: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 5:
Chain 5:  Elapsed Time: 0.93 seconds (Warm-up)
Chain 5:                0.604 seconds (Sampling)
Chain 5:                1.534 seconds (Total)
Chain 5:
```

```
# Summary model results
summary(fit_EXP_bayes)
```

```
 Family: gaussian
  Links: mu = identity
```

```
Formula: relative_yield ~ a - b * exp(-(log(b/(0.05 * a))/xs) * soil_test)
         a ~ 1
         b ~ 1
         xs ~ 1
   Data: data_example (Number of observations: 34)
  Draws: 5 chains, each with iter = 3000; warmup = 1000; thin = 1;
         total post-warmup draws = 10000

Regression Coefficients:
             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
a_Intercept     97.82      3.44    91.20   104.80 1.00     2700     4079
b_Intercept     24.26      5.73    14.71    36.08 1.00     3919     3472
xs_Intercept    25.36      9.31     8.07    44.34 1.00     2895     2717

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     7.07      0.93     5.55     9.14 1.00     4249     4986

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
# Extract posterior draws of xs
xs_EXP <- brms::as_draws_df(
  fit_EXP_bayes
)$b_xs_Intercept

# Calculate the posterior median and 95% credible interval
xs_EXP_hat <- median(xs_EXP)

xs_EXP_ci <- quantile(
  xs_EXP,
  probs = c(0.025, 0.975)
)

# Display the CSTV estimate and uncertainty
cat("\nBayesian Exponential (Mitscherlich) Model\n")
```

```
Bayesian Exponential (Mitscherlich) Model
```

```r
cat(
  "CSTV estimate:",
  round(xs_EXP_hat, 2),
  "\n"
)
```

```
CSTV estimate: 25.12
```

```r
cat(
  "95% Credible Interval:",
  round(xs_EXP_ci[1], 2),
  "-",
  round(xs_EXP_ci[2], 2),
  "\n"
)
```

```
95% Credible Interval: 8.07 - 44.34
```

4 Bayesian Model Averaging

Bayesian model averaging (BMA) combines the posterior distributions from the LP, QP, and exponential models. This approach incorporates uncertainty about which response model best describes the relationship between relative yield and soil-test phosphorus.

4.1 Estimate Marginal Likelihoods and Posterior Model Probabilities

Marginal likelihoods are estimated using bridge sampling. These values are used to calculate posterior model probabilities, assigning equal prior probabilities to the three candidate models.

```r
# Estimate marginal likelihoods using bridge sampling
bridge_LP <- bridgesampling::bridge_sampler(
  fit_LP_bayes,
  silent = TRUE
)

bridge_QP <- bridgesampling::bridge_sampler(
  fit_QP_bayes,
  silent = TRUE
)
```

```r
bridge_EXP <- bridgesampling::bridge_sampler(
  fit_EXP_bayes,
  silent = TRUE
)

# Calculate posterior model probabilities
model_prob <- bridgesampling::post_prob(
  bridge_LP,
  bridge_QP,
  bridge_EXP,
  prior_prob = c(1 / 3, 1 / 3, 1 / 3)
)

# Assign model names
names(model_prob) <- c("LP", "QP", "EXP")

# Display posterior model probabilities
model_prob
```

```
       LP        QP       EXP
0.2071636 0.2746554 0.5181809
```

The posterior model probabilities represent the relative support for each candidate model, conditional on the observed data, the specified priors, and the set of models considered.

4.2 Construct the Model-Averaged CSTV Posterior

The model-averaged posterior is constructed by sampling models according to their posterior probabilities and then sampling a CSTV draw from the selected model's posterior distribution.

```r
# Set the random seed for reproducibility
set.seed(555)

# Extract posterior CSTV draws from each model
draws_LP <- brms::as_draws_df(
  fit_LP_bayes
)[["b_xs_Intercept"]]

draws_QP <- brms::as_draws_df(
  fit_QP_bayes
```

```
)[["b_xs_Intercept"]]

draws_EXP <- brms::as_draws_df(
  fit_EXP_bayes
)[["b_xs_Intercept"]]

# Define the number of model-averaged posterior draws
n_draws <- 10000

# Sample a model for each draw according to its posterior probability
selected_model <- sample(
  names(model_prob),
  size = n_draws,
  replace = TRUE,
  prob = model_prob
)

# Initialize the model-averaged posterior distribution
draws_BMA <- numeric(n_draws)

# Sample CSTV draws from the selected model
draws_BMA[selected_model == "LP"] <- sample(
  draws_LP,
  sum(selected_model == "LP"),
  replace = TRUE
)

draws_BMA[selected_model == "QP"] <- sample(
  draws_QP,
  sum(selected_model == "QP"),
  replace = TRUE
)

draws_BMA[selected_model == "EXP"] <- sample(
  draws_EXP,
  sum(selected_model == "EXP"),
  replace = TRUE
)

# Retain the model-averaged CSTV posterior draws
theta_bma_bt <- draws_BMA
```

```r
# Summarize the model-averaged posterior
theta_bma_hat <- median(theta_bma_bt)

theta_bma_ci <- quantile(
  theta_bma_bt,
  probs = c(0.025, 0.975)
)

# Display the model-averaged CSTV estimate and credible interval
theta_bma_hat
```

```
[1] 21.95765
```

```r
theta_bma_ci
```

```
     2.5%     97.5%
 7.981501 42.918268
```

The resulting model-averaged posterior incorporates both parameter uncertainty within each model and uncertainty about which model generated the observed response pattern.

4.3 Results Table for CSTV

The results table summarizes the CSTV estimate and 95% credible interval for the model-averaged posterior and each individual model. The posterior model probabilities are also reported for the LP, QP, and exponential models.

```r
# Format CSTV estimates and 95% credible intervals
format_cstv <- function(estimate, interval) {
  sprintf(
    "%d [95%% CrI: %d-%d]",
    round(estimate),
    round(interval[1]),
    round(interval[2])
  )
}

# Assemble the CSTV results table
tab_CSTV <- data.frame(
  `Model Average` = format_cstv(
```

```
    theta_bma_hat,
    theta_bma_ci
  ),

  `Linear Plateau` = sprintf(
    "%s (W = %.2f)",
    format_cstv(xs_LP_hat, xs_LP_ci),
    unname(model_prob["LP"])
  ),

  `Quadratic Plateau` = sprintf(
    "%s (W = %.2f)",
    format_cstv(xs_QP_hat, xs_QP_ci),
    unname(model_prob["QP"])
  ),

  `Exponential` = sprintf(
    "%s (W = %.2f)",
    format_cstv(xs_EXP_hat, xs_EXP_ci),
    unname(model_prob["EXP"])
  ),

  check.names = FALSE
)

tab_CSTV
```

```
       Model Average                Linear Plateau
1 22 [95% CrI: 8-43] 15 [95% CrI: 7-31] (W = 0.21)
               Quadratic Plateau                    Exponential
1 22 [95% CrI: 10-43] (W = 0.27) 25 [95% CrI: 8-44] (W = 0.52)
```

5 Bayesian Calibration of Fertilizer Recommendations

5.1 Summarize the CSTV Posterior

The model-averaged Critical Soil Test Value (CSTV) is summarized using its posterior median and 95% credible interval. The median CSTV is used to calculate the phosphorus deficiency index.

```r
# Summarize the model-averaged CSTV posterior
cstv_median <- median(theta_bma_bt, na.rm = TRUE)

cstv_ci <- quantile(
  theta_bma_bt,
  probs = c(0.025, 0.975),
  na.rm = TRUE
)

cstv_median
```

```
[1] 21.95765
```

```r
cstv_ci
```

```
     2.5%     97.5%
 7.981501 42.918268
```

5.2 Calculate the Deficiency Index

The deficiency index represents the relative need for phosphorus fertilizer:

$$z_i = \max\left(0, 1 - \frac{STP_i}{CSTV_{\text{median}}}\right)$$

The index decreases to zero as soil-test P approaches or exceeds the median CSTV.

```r
# Calculate the phosphorus deficiency index
data_example <- data_example |>
  dplyr::mutate(
    z = pmax(0, 1 - soil_test / cstv_median)
  )
```

5.3 Estimate the Maximum Fertilizer Rate

This model estimates the maximum fertilizer rate, $P_{\max}$, from the observed optimum fertilizer rates and the phosphorus deficiency index, z_i.

$$OFR_i = P_{\max} z_i + \varepsilon_i$$

Here, OFR_i is the observed optimum fertilizer rate for observation i, $P_{\max}$ is the maximum fertilizer rate, z_i is the deficiency index, and ε_i is the residual error.

```
# Fit the model
fit_rec <- brms::brm(
  optimun_fertilizer_rate ~ 0 + z,
  data = data_example,
  family = gaussian(),
  prior = brms::set_prior(
    "normal(60, 30)",
    class = "b",
    lb = 0
  ),
  chains = 5,
  iter = 3000,
  warmup = 1000,
  cores = 1,
  seed = 555,
  save_pars = brms::save_pars(all = TRUE),
  control = list(
    adapt_delta = 0.99,
    max_treedepth = 15
  )
)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 4.1e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.41 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
```

```
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.077 seconds (Warm-up)
Chain 1:                0.129 seconds (Sampling)
Chain 1:                0.206 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.2e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.12 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.086 seconds (Warm-up)
Chain 2:                0.105 seconds (Sampling)
Chain 2:                0.191 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 1.1e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.11 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 3: Iteration:  300 / 3000 [ 10%]  (Warmup)
```

```
Chain 3: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 3: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 3: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 3: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 3: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 3: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 3: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 3: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 3: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 3: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.071 seconds (Warm-up)
Chain 3:                0.087 seconds (Sampling)
Chain 3:                0.158 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 1.4e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.14 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 4: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 4: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 4: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 4: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 4: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 4: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 4: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 4: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 4: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 4: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 4: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.066 seconds (Warm-up)
Chain 4:                0.088 seconds (Sampling)
Chain 4:                0.154 seconds (Total)
Chain 4:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 5).
Chain 5:
```

```
Chain 5: Gradient evaluation took 1.7e-05 seconds
Chain 5: 1000 transitions using 10 leapfrog steps per transition would take 0.17 seconds.
Chain 5: Adjust your expectations accordingly!
Chain 5:
Chain 5:
Chain 5: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 5: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 5: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 5: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 5: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 5: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 5: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 5: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 5: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 5: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 5: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 5: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 5:
Chain 5:  Elapsed Time: 0.081 seconds (Warm-up)
Chain 5:                0.082 seconds (Sampling)
Chain 5:                0.163 seconds (Total)
Chain 5:
```

```r
# Inspect model results
summary(fit_rec)
```

```
 Family: gaussian
  Links: mu = identity
Formula: optimun_fertilizer_rate ~ 0 + z
   Data: data_example (Number of observations: 34)
  Draws: 5 chains, each with iter = 3000; warmup = 1000; thin = 1;
         total post-warmup draws = 10000

Regression Coefficients:
  Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
z    38.69      4.85    29.38    48.43 1.00     4641     4043

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma    14.60      1.86    11.50    18.64 1.00     5036     4260

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
```

and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

5.4 Summarize the Maximum Fertilizer Rate

The posterior median and 95% credible interval of the regression coefficient summarize P_{max}.

```
# Extract posterior draws of Pmax
alpha_draws <- brms::as_draws_df(fit_rec)$b_z

# Summarize Pmax
Pmax_hat <- median(alpha_draws)

Pmax_ci <- quantile(
  alpha_draws,
  probs = c(0.025, 0.975)
)

Pmax_hat
```

```
[1] 38.67682
```

```
Pmax_ci
```

```
    2.5%    97.5%
29.38336 48.43206
```

5.5 Calculate Fertilizer Recommendations

The representative fertilizer recommendation is calculated using the median estimates of P_{max} and CSTV:

$$P_{\text{rec}}(STP) = P_{\text{max,median}} \max\left(0, 1 - \frac{STP}{CSTV_{\text{median}}}\right)$$

The recommendation range is calculated by substituting the lower and upper CSTV bounds into the equation while holding P_{max} at its median. The lower CSTV bound gives the upper recommendation, whereas the upper CSTV bound gives the lower recommendation.

This range reflects uncertainty in CSTV only. It is not a 95% credible interval for the fertilizer recommendation because uncertainty in P_{max} is not propagated.

```r
# Calculate the representative recommendation and its range
calculate_recommendation <- function(stp, pmax_rate, cstv) {
  pmax_rate * pmax(0, 1 - stp / cstv)
}

# Report the representative recommendation equation
cat(
  sprintf(
    "\nP_rec = %.2f * max(0, 1 - STP / %.2f)\n",
    Pmax_hat,
    cstv_median
  )
)
```

```
P_rec = 38.68 * max(0, 1 - STP / 21.96)
```

```r
# Report CSTV median and 95% credible interval
cat(
  sprintf(
    "CSTV: %.2f (95%% CrI: %.2f to %.2f)\n",
    cstv_median,
    cstv_ci[1],
    cstv_ci[2]
  )
)
```

```
CSTV: 21.96 (95% CrI: 7.98 to 42.92)
```

```r
# Report Pmax median and 95% credible interval
cat(
  sprintf(
    "Pmax: %.2f (95%% CrI: %.2f to %.2f)\n",
    Pmax_hat,
    Pmax_ci[1],
    Pmax_ci[2]
  )
)
```

```
Pmax: 38.68 (95% CrI: 29.38 to 48.43)
```

6 Summary of Results

6.1 Critical Soil Test Value

```
# Summarize CSTV estimates and model weights
tab_CSTV_summary <- data.frame(
  Model = c(
    "BMA",
    "Linear-Plateau",
    "Quadratic-Plateau",
    "Exponential"
  ),
  Median = c(
    theta_bma_hat,
    xs_LP_hat,
    xs_QP_hat,
    xs_EXP_hat
  ),
  Lower_95_CrI = c(
    theta_bma_ci[1],
    xs_LP_ci[1],
    xs_QP_ci[1],
    xs_EXP_ci[1]
  ),
  Upper_95_CrI = c(
    theta_bma_ci[2],
    xs_LP_ci[2],
    xs_QP_ci[2],
    xs_EXP_ci[2]
  ),
  Model_Weight = c(
    NA,
    model_prob["LP"],
    model_prob["QP"],
    model_prob["EXP"]
  )
)

knitr::kable(
  tab_CSTV_summary,
  digits = 2,
```

```
  col.names = c(
    "Model",
    "Median CSTV",
    "Lower 95% CrI",
    "Upper 95% CrI",
    "Model Weight"
  ),
  caption = "CSTV posterior summaries and model weights"
)
```

Table 1: CSTV posterior summaries and model weights

	Model	Median CSTV	Lower 95% CrI	Upper 95% CrI	Model Weight
	BMA	21.96	7.98	42.92	NA
LP	Linear-Plateau	15.46	7.22	31.10	0.21
QP	Quadratic-Plateau	21.90	9.95	42.65	0.27
EXP	Exponential	25.12	8.07	44.34	0.52

6.2 Fertilizer Recommendation

- **CSTV:** 22 [95% CrI: 8-43]
- **Pmax:** 39 [95% CrI: 29-48]

The final equation calculates the fertilizer recommendation using the soil-test phosphorus value (STP), the median CSTV, and the median maximum fertilizer rate P_{max}. To calculate a recommendation, substitute your soil-test P value and use the CSTV and P_{max} estimates reported above. If STP is equal to or greater than CSTV, the recommendation is zero.

$$P_{\text{rec}} = P_{\text{max}} \left(1 - \frac{STP}{CSTV}\right)$$